Milestone 4

Subscription page:

1. **Overview:**The interface consists of a top selector and a block of 4 plan cards.

   a. **Selector superior:** Radio buttons to choose between "Monthly" or "6 months" options.

   b. **Standard Plans (3 cards):**Their prices change dynamically depending on the option selected in step 1. The button leads to payment or login pages.

   c. **Institutional Plan (1 card): Fixed design. Does not display price and is not affected by the top selector.**Its button is a direct link to the contact form in the footer.

2. **Botón switch (radio buttons):** It allows you to select between 2 modalities that define the value to be deployed in each dynamic plan.

   a. Consists of 2 radio buttons.

   b. Crear un estado local: const [cycleDays, setCycleDays] = useState(30);

   c. Reference code:

```
<div className="flex justify-center gap-4 mb-10">
  <label className={`cursor-pointer px-6 py-2 rounded-full font-bold transition-colors
${cycleDays === 30 ? 'bg-blue-600 text-white' : 'bg-gray-200 text-gray-700'}`}>
    <input
      type="radio"
      name="planCycle"
      value={30}
      checked={cycleDays === 30}
      onChange={() => setCycleDays(30)}
className="hidden" // We hide the native input
    />
```

```
Monthly
  </label>

  <label className={`cursor-pointer px-6 py-2 rounded-full font-bold transition-colors
${cycleDays === 180 ? 'bg-blue-600 text-white' : 'bg-gray-200 text-gray-700'}`}>
    <input
      type="radio"
      name="planCycle"
      value={180}
      checked={cycleDays === 180}
      onChange={() => setCycleDays(180)}
      className="hidden"
    />
Semi-annual (Savings)
  </label>
</div>
```

d. **Dynamic Card Rendering:** The database will give you un json with plans in total (3 groups × 2 durations). Your job is not to display all 6 at once, but to display only 3 cards, and for the information within those 3 cards to change depending on whether the top switch is set to 30 or 180 days.

   i. **Create a local state:** const [plans, setPlans] = useState([]);

   ii. **Data collection:** GET /api/**subscription_plan**/?fields=**id, duration_days**, **price**, **plan_group**

   iii. **Reference code:**

```
export default function SubscriptionPage() {
  const router = useRouter();

// 1. Page states
  const [plans, setPlans] = useState([]);
  const [cycleDays, setCycleDays] = useState(30); // 30 (Mensual) o 180
(Semestral)

// 2. Data Fetching (Your exact code)
  useEffect(() => {
    const fetchPlans = async () => {
      const { data, error } = await supabase
        .from('subscription_plan')
        .select('id, duration_days, price, plan_group');
      if (!error) setPlans(data);
    };
    fetchPlans();
  }, []);
```

iv. **Example of a return structure:**

id	duration_days	price	plan_group
7	180	300000	institucional
1	30	40000	particular
2	180	200000	particular
3	30	50000	derivacion
4	180	250000	derivacion
5	30	30000	aburrimiento
6	180	150000	aburrimiento

v. **Keys to identify each group:**['particular', 'derivacion', 'aburrimiento']. It is obtained from the field **'plan_group'**

vi. **Plan rendering:**Filter the array '**plans**' looking for the corresponding 'plan_group' that matches 'duration_days' === cycleDays.

e. **Subscribe button:**This button redirects to the next page under the following conditions:

i. You must verify if the user is logged in:

1. If you are logged in,

   a. Redirect to the payment page using the plan ID: `/checkout?planId=PLAN_ID`

2. If you are not logged in, redirect to the login page: /login?redirectTo=/checkout?planId=PLAN_ID

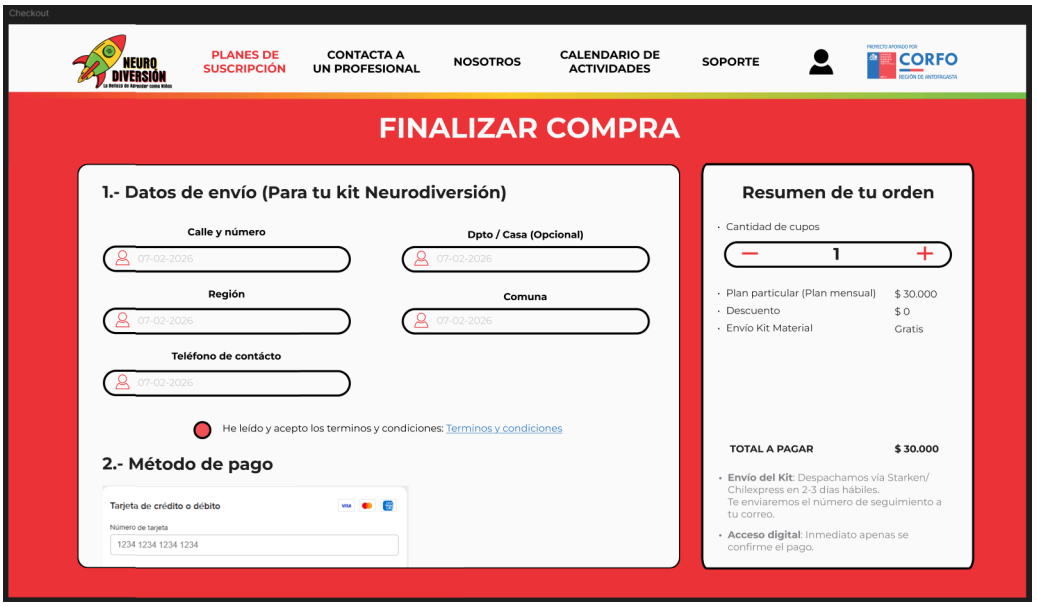
   a. **Modification to the login page:**If the user enters the login data, "get_app_context()" should not be called

immediately without first checking if there is a redirect variable in the URL (This has priority)

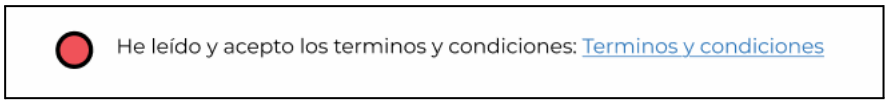
b. If the user does not have an account and presses “Create account”, the parameter should be passed again via the URL:/signup?redirectTo=/checkout?planId=PLAN_ID

Checkout page: It displays a form for entering personal data, confirmation of having read the terms and conditions, the payment method which corresponds to the integration with the payment gateway, and the order summary.

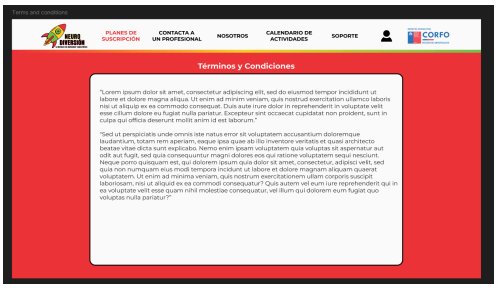
1. **Load information:**You must obtain the plan ID from the URL and retrieve the plan from the database:GET /API/subscription_plan/PLAN_ID?fields=**name**, **duration_days**, **price**, **mp_preapproval_plan_id**, **discount_percentage**



a. Terms and conditions: Displays a radio button and a hyperlink to the terms and conditions page:



i. Terms and conditions page:



b. **Order summary:**It consists of a component that allows increasing the plan's quotas, a plan description, and the total amount to pay.

i. **Increased quotas:**

ii. **Plan description:**

iii. **Total value:**

1. **This corresponds to the value of 'price' multiplied by the number of 'slots' selected and minus the 'discount_percentage'**